

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



FINAL REPORT

Massive Data Processing Techniques In Data Science

Lecturer: **M.Sc. NGUYEN THANH AN**

Authors: **PHAM PHU BINH – 521H0495**

NGUYEN KHAC HUY – 521H0502

GIANG HOANG DAT – 521H0498

DO HOANG DUY – 521H0395

Class : **21H50301**

Group : **8**

Academic year : **25**

HO CHI MINH CITY, 2024

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



FINAL PROJECT

Massive Data Processing Techniques In Data Science

Lecturer: **M.Sc. NGUYEN THANH AN**

Authors: **PHAM PHU BINH – 521H0495**

NGUYEN KHAC HUY – 521H0502

GIANG HOANG DAT – 521H0498

DO HOANG DUY – 521H0395

Class : **21H50201**

Group : **8**

Course : **25**

HO CHI MINH CITY, 2024

ACKNOWLEDGEMENT

To complete this final project, we would like to express our gratitude to Ton Duc Thang University for providing the necessary facilities and to the teachers who have supported us throughout our studies at the TDT University. Moreover, we love to give thanks to our parents who gave us many chances to approach knowledge at TDTU and motivated, supported us a lot.

Especially, we would like to thank McS. Nguyen Thanh An for teaching us with great dedication and detail, so that we have enough knowledge that we used for this essay. Due to our limited experience and acknowledgement, we make sure that there are some mistakes in our work. We sincerely hope to receive feedbacks and constructive criticism from teacher who instructed us. So that we can complete this essay more effectively.

We would like to express our heartfelt thanks and wishes teacher good health.

THE PROJECT IS COMPLETED AT TON DUC THANG UNIVERSITY

I hereby declare that this is the product of our project and is guided by Nguyen Thanh An. The research contents and results in this topic are honest and have not been published in any forms before. The data in the tables for analysis, comments, and evaluation were collected by the author from different sources and clearly stated in the reference section. In addition, the project also uses a number of comments, assessments as well as data from other authors and other organizations, all with citations and source notes. If any fraud is discovered, I will take full responsibility for the content of my project. Ton Duc Thang University is not involved in copyright violations caused by me during the implementation process (if any).

Ho Chi Minh City, 24th May 2024

Authors

(Signatures and full names)

Pham Phu Binh

Nguyen Khac Huy

Giang Hoang Dat

Do Hoang Duy

SUMMARY

This final project for the course “Massive Data Processing Techniques in Data Science” showcases our comprehensive approach to handling and analyzing large datasets using advanced machine learning algorithms in PySpark. Each task involves meticulous data preprocessing tailored to the specific requirements, followed by the application of various algorithms including KMeans, ALS, SVD, Linear Regression, Multi-class Perceptron, Random Forest, and Linear Support Vector Machine.

The project not only delves into the theoretical underpinnings of each algorithm but also provides detailed insights into the practical implementation and outcomes. To enhance the interpretability of our findings, we leverage matplotlib.pyplot for effective data visualization. This summary encapsulates our methodical journey through preprocessing, algorithmic execution, and result visualization, culminating in a robust analysis of massive data processing techniques

TASK ASSIGNMENT

Student name	Student id	Task assignment	Process
Phạm Phú Bình	521H0495	Task 1, Task 6	100%
Nguyễn Khắc Huy	521H0502	Task 4, Task 6	100%
Giang Hoàng Đạt	521H0498	Task 3, Task 6	100%
Đỗ Hoàng Duy	521H0395	Task 2, Task 5, Task 6	100%

TABLE OF CONTENTS

ACKNOWLEDGEMENT	1
THE PROJECT IS COMPLETED AT TON DUC THANG UNIVERSITY	2
SUMMARY	3
TASK ASSIGNMENT	4
TABLE OF CONTENTS	5
TASK 1 – Clustering.....	7
1.1 Data	7
1.2 Algorithm: Kmeans to clustering.....	8
1.3 Result	11
TASK 2 – Dimensionality Reduction with SVD	11
2.1 Introduction.....	11
2.2 Explain code.....	15
2.3 Results.....	19
TASK 3 – Recommendation with Collaborative Filtering	20
3.1 Dataset.....	20
3.2 Algorithm	21
3.3 Result.....	22
TASK 4 – Stock price regression.....	22
4.1. Understanding Dataset	22
4.2. Linear Regression with MSE	23
4.3. Train linear regression model to predict price of next day with dataset step – by – step.	24
TASK 5 – Multi-class classification	30
5.1 Algorithm	30
5.2 Result.....	31

REFERENCES.....	33
-----------------	----

TASK 1 – Clustering

1.1 Data

Data sets	Description
mnist_mini.csv	Hand-written digit images in the MNIST data set. 10000 data rows. Each row has 785 integers. • The first number: digit kind (0, 1, 2, 3, ..., 9) • Remaining 784 numbers: pixels of grayscale images (28 x 28).

Preprocessing data: Assign weight in line [0, 1, 2, 3, 4, 7, 8, 11, 18, 61]

```

*** +-----+-----+-----+-----+
    |label|           features|weight|
    +-----+-----+-----+-----+
    | 7|[0.0,0.0,0.0,0.0,...| 100.0|
    | 2|[0.0,0.0,0.0,0.0,...| 100.0|
    | 1|[0.0,0.0,0.0,0.0,...| 100.0|
    | 0|[0.0,0.0,0.0,0.0,...| 100.0|
    | 4|[0.0,0.0,0.0,0.0,...| 100.0|
    | 1|[0.0,0.0,0.0,0.0,...|  1.0|
    | 4|[0.0,0.0,0.0,0.0,...|  1.0|
    | 9|[0.0,0.0,0.0,0.0,...| 100.0|
    | 5|[0.0,0.0,0.0,0.0,...| 100.0|
    | 9|[0.0,0.0,0.0,0.0,...|  1.0|
    | 0|[0.0,0.0,0.0,0.0,...|  1.0|
    | 6|[0.0,0.0,0.0,0.0,...| 100.0|
    | 9|[0.0,0.0,0.0,0.0,...|  1.0|
    | 0|[0.0,0.0,0.0,0.0,...|  1.0|
    | 1|[0.0,0.0,0.0,0.0,...|  1.0|
    | 5|[0.0,0.0,0.0,0.0,...|  1.0|
    | 9|[0.0,0.0,0.0,0.0,...|  1.0|
    | 7|[0.0,0.0,0.0,0.0,...|  1.0|
    | 3|[0.0,0.0,0.0,0.0,...| 100.0|

```

Data sets after assign weight

1.2 Algorithm: Kmeans to clustering

Clustering:

Cluster analysis is a technique used in data mining and machine learning to group similar objects into clusters. K-means clustering is a widely used method for cluster analysis where the aim is to partition a set of objects into K clusters in such a way that the sum of the squared distances between the objects and their assigned cluster mean is minimized. To summary, Clustering is the process of dividing the entire data into groups (also known as clusters) based on the patterns in the data. For example, a bank wants to give credit card offers to its customers. Instead of looking at each individual and making strategic decisions for each one, the bank divides them into three groups to easily make strategic decisions for each group.



Different group that need to divide

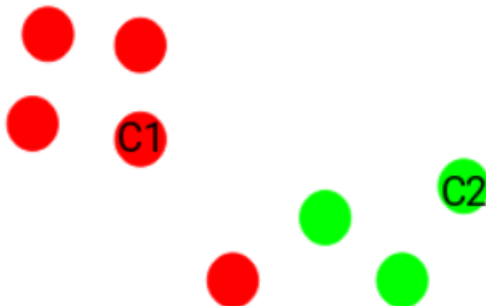
Kmeans:

- **Definition:** is a centroid-based algorithm or a distance-based algorithm, where we calculate the distances to assign a point to a cluster. In K-Means, each cluster is associated with a centroid.

- **Objective:** to minimize the sum of distances between the points and their respective cluster centroid. Here is some example:
 - Grouping similar data points: K-means aims to identify patterns in your data by grouping data points that share similar characteristics together. This allows you to discover underlying structures within the data.
 - Minimizing within-cluster distance: The algorithm strives to make sure data points within a cluster are as close as possible to each other, as measured by a distance metric (usually Euclidean distance). This ensures tight-knit clusters with high cohesiveness.
 - Maximizing between-cluster distance: Conversely, k-means also tries to maximize the separation between clusters. Ideally, data points from different clusters should be far apart, making the clusters distinct from each other.

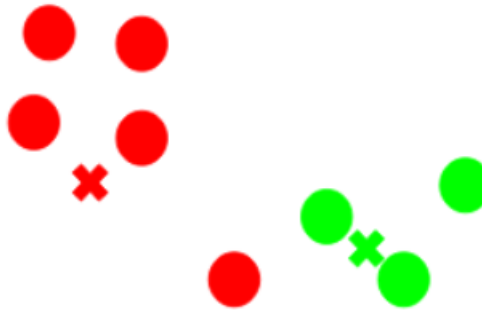
Kmeans work:

1. Initialization: Start by randomly selecting K points from the dataset. These points will act as the initial cluster centroids.
2. Assignment: For each data point in the dataset, calculate the distance between that point and each of the K centroids. Assign the data point to the cluster whose centroid is closest to it. This step effectively forms K clusters.



Assign all the points to the closest cluster centroid

3. Update centroids: Once all data points have been assigned to clusters, recalculate the centroids of the clusters by taking the mean of all data points assigned to each cluster.



Update new centroid

4. Repeat: Repeat steps 2 and 3 until convergence. Convergence occurs when the centroids no longer change significantly or when a specified number of iterations is reached.

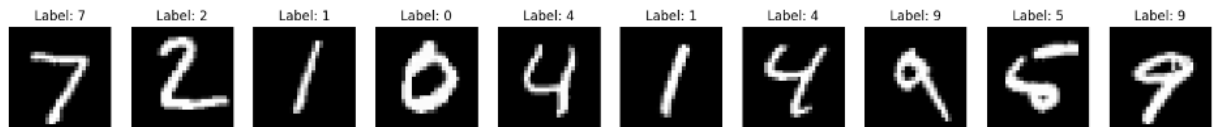


Repeat step 2 and 3

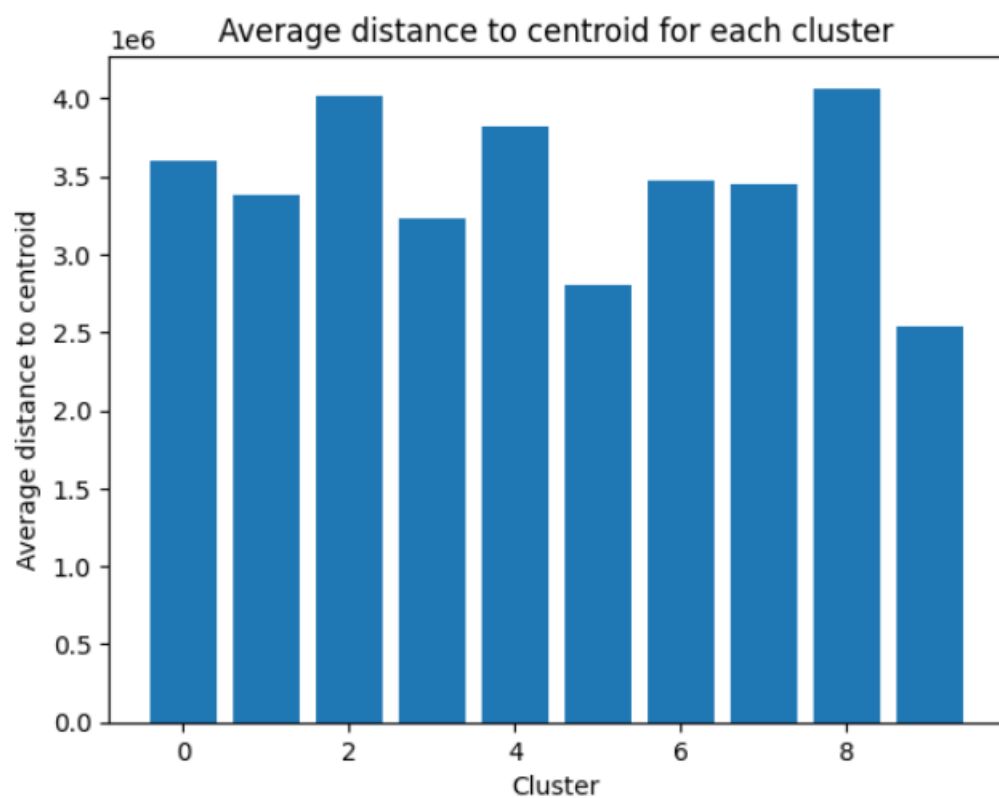
5. Final Result: Once convergence is achieved(meet the criteria), the algorithm outputs the final cluster centroids and the assignment of each data point to a cluster. Here is some stopping criteria for K-Means Clustering:
 - Centroids of newly formed clusters do not change
 - Points remain in the same cluster

- Maximum number of iterations is reached

1.3 Result



Visualize dataset



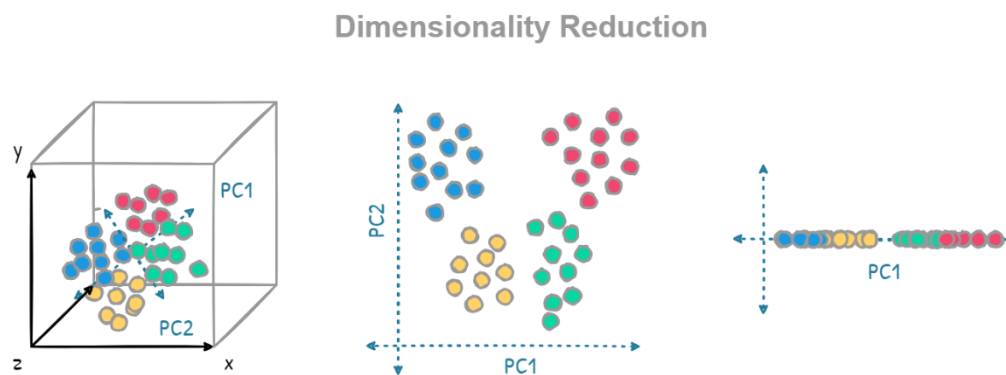
Visualize the average distance to centroid for each cluster

TASK 2 – Dimensionality Reduction with SVD

2.1 Introduction

Dimensionality reduction is a machine learning and data analysis technique in which data is transformed from a high-dimensional to a low-dimensional environment. The goal is to maintain as much meaningful information as feasible while minimizing the number of features (or dimensions) in the dataset. This can be done for a variety of purposes, including:

- Reducing the complexity of a model.
- Improving the performance of a learning algorithm.
- Making it easier to visualize the data.

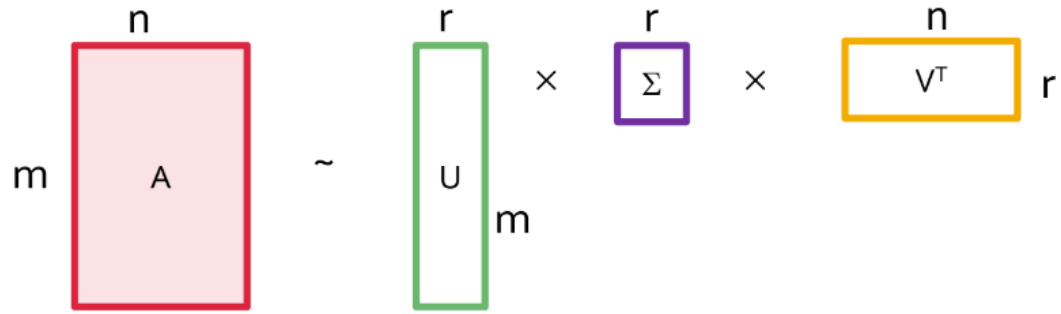


Common Dimensionality Reduction Techniques

1. Principal Component Analysis (PCA):
 - PCA is a linear technique that transforms the data into a new coordinate system such that the greatest variances by any projection of the data come to lie on the first coordinates (called principal components), the second greatest variances on the second coordinates, and so on.
2. Singular Value Decomposition (SVD):

- SVD is a matrix factorization technique that decomposes a matrix into three other matrices: U , Σ and V . It is particularly useful for reducing the dimensionality of large datasets and is closely related to PCA.
3. t-Distributed Stochastic Neighbor Embedding (t-SNE):
 - t-SNE is a non-linear technique primarily used for visualization. It is effective for reducing data to 2 or 3 dimensions for visualization purposes by preserving the distances between points in high-dimensional space.
 4. Linear Discriminant Analysis (LDA):
 - LDA is both a dimensionality reduction and a classification technique. It projects the data in a way that maximizes the separation between multiple classes.
 5. Autoencoders:
 - Autoencoders are neural network-based techniques for unsupervised learning of efficient codings. They can be used for dimensionality reduction by training the network to compress data to a lower-dimensional latent space and then reconstruct it.

Dimensionality reduction using Singular Value Decomposition (SVD) allows us to reduce the number of dimensions of a dataset while retaining the most important information.



SVD is a mathematical technique that decomposes a matrix A into three other matrices: U , Σ , and V^T

$$A = U\Sigma V^T$$

A : Input matrix ($m \times n$)

U : Left matrix ($m \times r$). Its columns are the left singular vectors of A

V : Right matrix ($n \times r$). Its rows are the right singular vectors of A

Σ : Diagonal matrix ($r \times r$)

- U, Σ, V : unique
- U, V : column orthonormal (trực chuẩn)
- $U^T \cdot U = I, V^T \cdot V = I$ (I : identity matrix)
- Σ : singular values are non-negative, and sorted in decreasing order

Benefits and Applications

- Reduced Storage and Computation: Lower-dimensional representations take up less storage space and processing resources.
- Noise Reduction: SVD can assist filter out noise while maintaining the most important elements of the data.

- Visualization: Reducing dimensions to two or three allows for more straightforward representation of complex data.
- Feature Extraction: Assists in determining the most significant features or components of data.

2.2 Explain code

Part 1: Dimensionality reduction

1. Initialization:
 - Initializes a Spark session.
 - Sets the file paths for the input dataset and output SVD results.
2. read_data:
 - Uses PySpark's CSV reader to load the dataset into a DataFrame.
 - Processing:
 - The first column (which is the label) is renamed to label.
 - An id column is added to uniquely identify each row, which will be useful for joining purposes later.
3. prepare_data:
 - Uses VectorAssembler from PySpark's MLlib.
 - Purpose: Combines all feature columns (pixels in the MNIST dataset) into a single vector column named features. This step prepares the data for matrix operations required by SVD.
 - Processing: Converts the individual pixel columns into a single vector column.
4. compute_svd:

- Uses RowMatrix from PySpark's MLlib to perform SVD.
- Purpose: Reduces the dimensionality of the feature vectors from 784 dimensions to 3 dimensions.
- Processing:
 - Converts the feature vectors to a format suitable for RowMatrix. This involves transforming each row of features into a dense vector.
 - Creates a RowMatrix object from the RDD of dense vectors.
 - Computes the Singular Value Decomposition (SVD) of the RowMatrix. The computeSVD function decomposes the matrix into three components: U (left singular vectors), S (singular values), and V (right singular vectors).
 - Extracts the U matrix, which contains the reduced dimensionality vectors.
 - Converts the U matrix into a DataFrame, retaining the id column for later joining.

5. save_svd_result:

- Purpose: Saves the reduced dimensionality data to a CSV file for later use.
- Processing:
 - The DataFrame is coalesced into a single partition to ensure it is saved as a single CSV file.
 - The DataFrame is then saved to the specified output path as a CSV file.

6. run:

- Orchestrates the entire process by calling the appropriate methods in sequence: reading data, preparing it, computing SVD, and saving the results.
- RowMatrix and SVD: The code uses PySpark's RowMatrix and computeSVD to perform Singular Value Decomposition. This step is crucial as it decomposes the original high-dimensional data into lower-dimensional space while preserving the essential structure of the data.
- VectorAssembler: This is used to combine the feature columns into a single vector column, which is a prerequisite for creating a RowMatrix.
- Adding id Column: The id column is added to maintain a reference for each data point, which will be useful when joining this data with the clustering results in Part 2
- Coalescing and Saving: The coalescing step ensures that the SVD result is saved as a single CSV file rather than a directory of files, making it easier to load and use in Part 2.

Part 2: Visualize

1. Initialization:
 - Initializes a Spark session.
 - Sets the file paths for the input SVD results and output visualization.
2. read_data:
 - The SVD results are read from a CSV file into a Spark DataFrame.
 - The clustering results are also read into a Spark DataFrame.
 - Both data frames are returned for further processing.

3. sample_data:

- The SVD results are randomly sampled to select 100 data points. This is achieved using Spark's ``rand()`` function to shuffle the data and ``limit(100)`` to select the top 100 rows

4. combine_data

- Read the clustering results from Task 1.
- The sampled data is combined with the clustering results using the ``id`` column as the join key. This ensures that each sampled data point retains its cluster assignment from Task 1.

5. visualize_data

- The combined data is converted to a Pandas DataFrame to facilitate easy plotting with Matplotlib.
- A 3D scatter plot is created using Matplotlib's ``scatter`` function.
- Different clusters are represented using different colors, and a legend is added to indicate the clusters.
- Axis labels and a title are added for clarity.

6. run

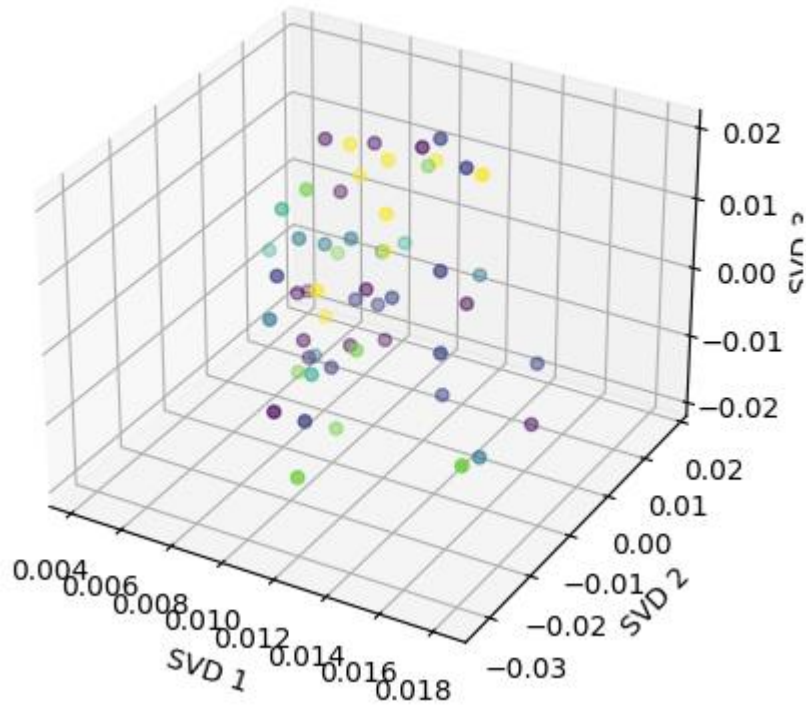
- Orchestrates the entire process by calling the appropriate methods in sequence: read data, sample data, combine with clustering results, and visualize.

2.3 Results

```
+---+-----+-----+-----+
| id|          svd1|          svd2|          svd3|
+---+-----+-----+-----+
| 0|0.006691310321257989|0.004439054715611977|0.013516943999505298|
| 1|0.008501569102646569|-0.00249999658374...|-0.01941258381501896|
| 2|0.004105904934289446| 0.01023705818203932|-0.00896155777650...|
| 3|0.014058171614146636|-0.01828164590075...|0.001806857741902...|
| 4|0.006777044890823185|-0.00771510613675...|0.014367324540265743|
| 5|0.005869354573327854|0.015312737168598316|-0.01016832680847...|
| 6| 0.00740645573953409|0.004469591332743806| 0.01071474599251274|
| 7|0.007744236916142777|0.005212505402246573|0.006261580736210...|
| 8|0.010326327418547089|-0.00430556904331...|0.003738439305364357|
| 9|0.011735611970446782|0.006229305083540...|0.015974881488080322|
|10|0.008960422979592209|-0.02370953777481...|-0.01121121080704598|
|11|0.008958264922108524|-0.00866613275016...|-0.00350541369519...|
|12|0.009528890426201442|0.006533932522156126| 0.01941262413902697|
|13| 0.01005008540836874|-0.01968798091415718|0.001135253208046...|
|14|0.007006768483532377| 0.01407458333545101|-0.00867097057940...|
|15|0.006144308369017278|-0.00668506060580...|-0.00805798020422...|
|16|0.008830763683010947|-0.00249326520987...| 0.01793051593963947|
|17|0.007829761155399564|-3.28863462145109...|0.011988553245534581|
|18|0.009784842633203705|-3.36729700939276...|-0.00742561075189...|
|19|0.006716434715901316|0.002728686858968...|0.013127816886947717|
+---+-----+-----+-----+
only showing top 20 rows
```

Dataframe present svd1, svd2, svd3

3D Visualization of Clustered Data Points



Visualize clustered data points

TASK 3 – Recommendation with Collaborative Filtering

3.1 Dataset

Collaborative filtering is a popular method used in recommendation systems to suggest items to users based on the preferences and behaviors of other users. It operates on the principle that if users agree on some items, they are likely to agree on others as well. Collaborative filtering can be divided into two main categories: user-based and item-based

In this task, the performance of an Alternating Least Squares (ALS) model will be evaluated using PySpark, focusing on the Mean Squared Error (MSE) and the impact of varying the number of similar users. The dataset used will be **'ratings2k.csv'**.

Load and split the dataset '**ratings2k.csv**' into a 7:3 ratio; train the ALS model with ranks from 10 to 20; evaluate performance using Mean Squared Error (MSE); visualize the correlation between MSE values and the number of similar users with a bar chart.

3.2 Algorithm

Implementation Steps

1. Data Loading and Splitting:

- Create a DataLoader class to handle loading the dataset from ratings2k.csv.
- Split the dataset into training and test sets with a 70:30 ratio use randomSplit

2. Model Training:

- Implement an ALSModel class to train the ALS model.
- Train the model with ranks (number of similar users) ranging from 10 to 20.

3. Model Evaluation:

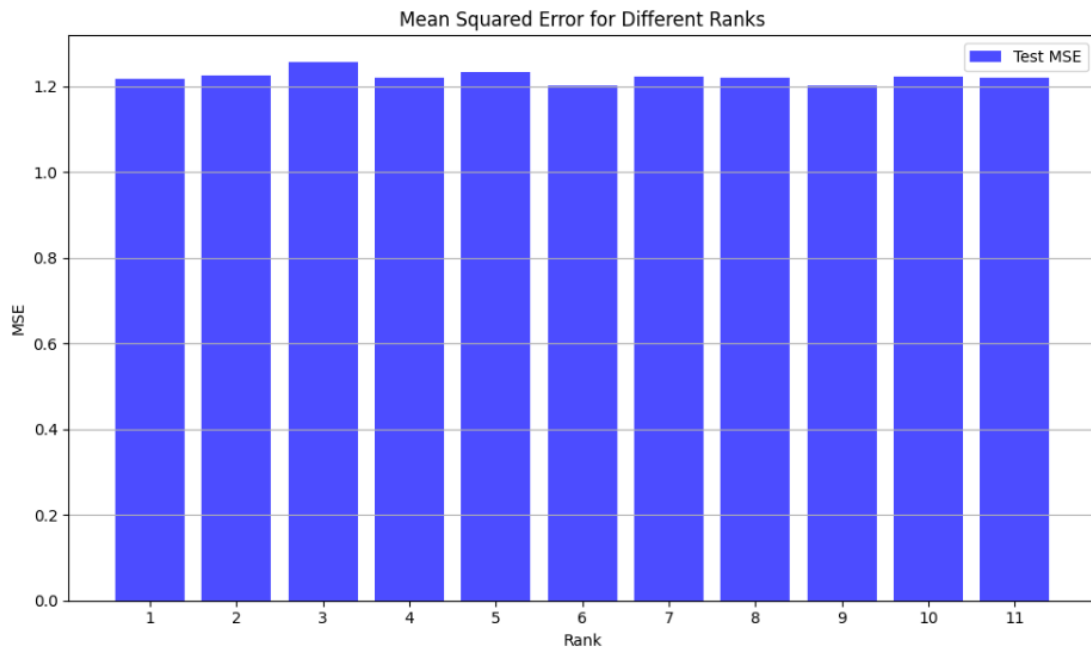
- Within the ALSModel class, evaluate the model using Mean Squared Error (MSE) for each rank.
- Store the MSE results for each rank.

4. Visualization:

- Create a Visualization class to plot the results.
- Use a bar chart to visualize the correlation between the number of similar users (rank) and the MSE values.

3.3 Result

```
Rank: 10, MSE: 1.217541550353672
Rank: 11, MSE: 1.2257129790911045
Rank: 12, MSE: 1.2554500692926422
Rank: 13, MSE: 1.2191499414384535
Rank: 14, MSE: 1.2316251660184723
Rank: 15, MSE: 1.2010920298205943
Rank: 16, MSE: 1.2211021784013907
Rank: 17, MSE: 1.2203811295509794
Rank: 18, MSE: 1.201800285509657
Rank: 19, MSE: 1.2236828586518593
Rank: 20, MSE: 1.2194184952093328
```



Mean Squarred Error for dirrenrent Ranks

TASK 4 – Stock price regression

4.1. Understanding Dataset

In this section, we receive a dataset named “*stockHVN2022.csv*” this is the Stock prices of HVN code in HOSE in 2022 (until Nov 18th).

This dataset consist header with two columns, one is Ngay mean date with String type and the remaining column named HVN includes prices with Double type.

The dataset has 220 lines include header, the dataset's structure below:

```
+-----+-----+
|      Ngay|  HVN|
+-----+-----+
|2022-11-18|  9.3|
|2022-11-17|9.42|
|2022-11-16|9.21|
|2022-11-15|8.61|
|2022-11-14|9.25|
+-----+-----+
only showing top 5 rows

root
 |-- Ngay: date (nullable = true)
 |-- HVN: double (nullable = true)
```

dataset structure and schema after convert String into date type

4.2. Linear Regression with MSE

Regression is the statistical approach to find the relationship between variables. The Linear Regression assumes a linear relationship between variables. Depending on the number of input variables, the regression problem classified into two types:

- Simple linear regression.
- Multiple linear regression.

In this task because of the requirement, feature column of model should be a vector consists of numbers so that we use multiple linear regression to handle.

Simple expression of multiple linear regression. Multiple linear regression is used to estimate the relationship between two or more independent variables and one dependent variable. You can use multiple linear regression when you want to know:

- How strong the relationship is between two or more independent variables and one dependent variable
- The value of the dependent variable at a certain value of the independent variables (e.g. the expected yield of a crop at certain levels of rainfall, temperature, and fertilizer addition).

Linear regression formula:

$$y = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n + \varepsilon$$

y = predicted value of the dependent variable.

β_0 = the y-intercept (value of y when all other parameters are set to 0).

$\beta_n X_n$ = the regression coefficient of the last independent variable.

ε = model error.

In this section, we use MSE (Mean Square Error) to calculate the error for this section. Here is simple expression for MSE. MSE is the average squared error between actual and predicted values.

Squared error, also known as L2 loss, is a row-level error calculation where the difference between the prediction and the actual is squared. MSE is the aggregated mean of these errors, which helps us understand the model performance over the whole dataset.

The main draw for using MSE is that it squares the error, which results in large errors being punished or clearly highlighted. It's therefore useful when working on models where occasional large errors must be minimised.

The formula to calculate MSE:

$$MSE = \frac{\sum(actual\ value - prediction\ value)^2}{number\ of\ observation}$$

4.3. Train linear regression model to predict price of next day with dataset step – by – step.

Here is the pseudocode for finishing the task:

```

start a spark session 'StockPricePrediction'

# prepare data class
class data_preparation:

    load_data function:

        load data from csv file and convert String type into date type in
        col('Ngay')

    split_data_function (train_test_split):

        split rows from before Jun (30-6-2022) into train_df
        store remaining rows into test_df

    create_fluctuation function:

        # with the given formula in requirement
        create new cols to store price of date[k] and [k-1]
        define col 'fluctuation' store fluctuation of current row

    create_feature function:

        create new five cols to store fluctuation of 5 previous dates
        add it into new col array in col('range_5_previous_dates')
        convert array into vector
        add range of next day into col 'today_range' of the current row

# Training class
class TrainingModel:

    train_model function:

        get df after create_feature() to fit model (use model in spark lib)
        create a variable to store mse of the process

```

evaluate_model function:

same with train_model() without fit step

plot_mse function:

get train_mse and test_mse from two previous defs

add into a array

visualize two MSEs by matplotlib

Firstly, we load data from .csv file from google drive to process. Load and print down to analyze detail.

```
+-----+-----+
| Ngay      | HVN  |
+-----+-----+
| 18/11/2022 | 9.3  |
| 17/11/2022 | 9.42 |
| 16/11/2022 | 9.21 |
| 15/11/2022 | 8.61 |
| 14/11/2022 | 9.25 |
| 11/11/2022 | 9.9   |
| 10/11/2022 | 9.76 |
| 09/11/2022 | 10.4  |
| 08/11/2022 | 10.25 |
| 07/11/2022 | 10.1  |
| 04/11/2022 | 10.7  |
| 03/11/2022 | 10.85 |
| 02/11/2022 | 10.9  |
| 01/11/2022 | 10.9  |
| 31/10/2022 | 10.8  |
| 28/10/2022 | 10.75 |
| 27/10/2022 | 10.85 |
| 26/10/2022 | 10.55 |
| 25/10/2022 | 10.55 |
| 24/10/2022 | 10.55 |
+-----+-----+
only showing top 20 rows

root
|-- Ngay: string (nullable = true)
|-- HVN: double (nullable = true)
```

dataset form and it's schema

Then, we split it into two data frames named train and split. One to train out model to predict the next day price HVN and one to evaluate.

Train data:

```
+-----+-----+
|      Ngay|  HVN|
+-----+-----+
|2022-06-30| 15.5|
|2022-06-29|16.05|
|2022-06-28| 16.5|
|2022-06-27| 16.3|
|2022-06-24|15.25|
+-----+-----+
```

only showing top 5 rows

Test data:

```
+-----+-----+
|      Ngay|  HVN|
+-----+-----+
|2022-11-18| 9.3|
|2022-11-17|9.42|
|2022-11-16|9.21|
|2022-11-15|8.61|
|2022-11-14|9.25|
+-----+-----+
```

only showing top 5 rows

train dataframe and test dataframe after splitting

We calculate the fluctuation range depend on k days previous with given formula. And store in a new column named “fluctuation”. Because the dataset arrange date from the latest date to earlier so we just solve top down for more easily. Note that the first date of the data mean the last row the data should be 0.0 of fluctuation.

Train dataframe after add fluctuation col

Ngay	HVN	fluctuation
2022-06-30	15.5	0.024157729390098533
2022-06-29	16.05	0.02424991934616621
2022-06-28	16.5	0.024989320803075588
2022-06-27	16.3	0.02544259514470476
2022-06-24	15.25	0.02507886435331233
2022-06-23	15.3	0.02501042100875365
2022-06-22	15.35	0.024888980687803396
2022-06-21	14.95	0.025280180134077117
2022-06-20	14.6	0.025283745439805477
2022-06-17	15.6	0.025272025272025316

only showing top 10 rows

Test dataframe after add fluctuation col

Ngay	HVN	fluctuation
2022-11-18	9.3	0.02491819783538891
2022-11-17	9.42	0.02518703241895258
2022-11-16	9.21	0.02676086491115393
2022-11-15	8.61	0.027430934279714166
2022-11-14	9.25	0.027308798252236907
2022-11-11	9.9	0.027369656966966013
2022-11-10	9.76	0.026818901836545887
2022-11-09	10.4	0.02643615784169308
2022-11-08	10.25	0.027087170042971113
2022-11-07	10.1	0.02716236722306522

only showing top 10 rows

Train and test data frame after have fluctuation column with $k = 50$

In the next step, a new data frame is create with two columns “range_5_previous_dates” and “today_range” store vectors of 5 previous dates previous fluctuation range, the range of next day of current row respectively.

range_of_5_previous_dates	today_range
[0.02424991934616621,0.024989320803075588,0.02544259514470476,0.02507886435331233,0.02501042100875365]	0.0
[0.024989320803075588,0.02544259514470476,0.02507886435331233,0.02501042100875365,0.024888980687803396]	0.024157729390098533
[0.02544259514470476,0.02507886435331233,0.02501042100875365,0.024888980687803396,0.025280180134077117]	0.02424991934616621
[0.02507886435331233,0.02501042100875365,0.024888980687803396,0.025280180134077117,0.025283745439805477]	0.024989320803075588
[0.02501042100875365,0.024888980687803396,0.025280180134077117,0.025283745439805477,0.025272025272025316]	0.02544259514470476
[0.024888980687803396,0.025280180134077117,0.025283745439805477,0.025272025272025316,0.024733051899677155]	0.02507886435331233
[0.025280180134077117,0.025283745439805477,0.025272025272025316,0.024733051899677155,0.02467008075635213]	0.02501042100875365
[0.025283745439805477,0.025272025272025316,0.024733051899677155,0.02467008075635213,0.024309284389339145]	0.024888980687803396
[0.025272025272025316,0.024733051899677155,0.02467008075635213,0.024309284389339145,0.02412556922778809]	0.025280180134077117
[0.024733051899677155,0.02467008075635213,0.024309284389339145,0.02412556922778809,0.024136936243869513]	0.025283745439805477

only showing top 10 rows

feature train dataframe with vector range of 5 previous dates with $k = 50$

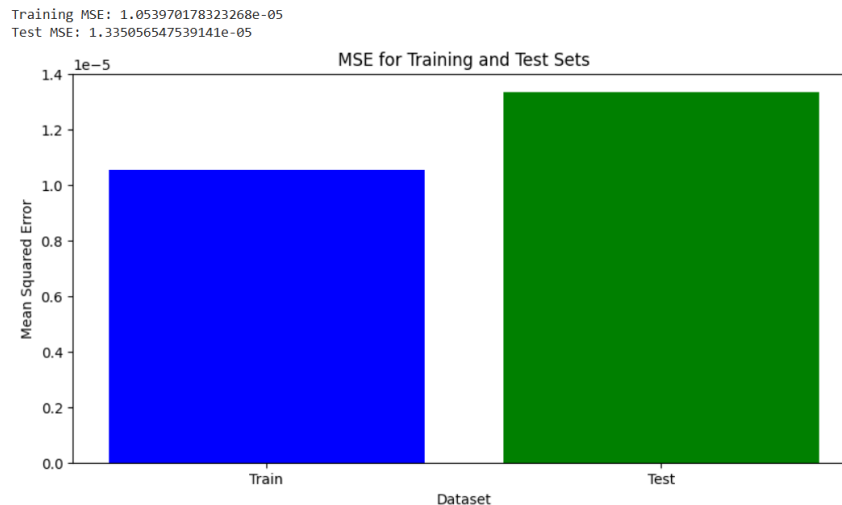
range_of_5_previous_dates	today_range
[0.02518703241895258,0.02676086491115393,0.027430934279714166,0.027308798252236907,0.027369656966966013]	0.0
[0.02676086491115393,0.027430934279714166,0.027308798252236907,0.027369656966966013,0.026818901836545887]	0.02491819783538891
[0.027430934279714166,0.027308798252236907,0.027369656966966013,0.026818901836545887,0.02643615784169308]	0.02518703241895258
[0.027308798252236907,0.027369656966966013,0.026818901836545887,0.02643615784169308,0.027087170042971113]	0.02676086491115393
[0.027369656966966013,0.026818901836545887,0.02643615784169308,0.027087170042971113,0.02716236722306522]	0.027430934279714166
[0.026818901836545887,0.02643615784169308,0.027087170042971113,0.02716236722306522,0.02684463107378521]	0.027308798252236907
[0.02643615784169308,0.027087170042971113,0.02716236722306522,0.02684463107378521,0.0258902077151336]	0.027369656966966013
[0.027087170042971113,0.02716236722306522,0.02684463107378521,0.0258902077151336,0.025565677343520388]	0.026818901836545887
[0.02716236722306522,0.02684463107378521,0.0258902077151336,0.025565677343520388,0.025323824770775695]	0.02643615784169308
[0.02684463107378521,0.0258902077151336,0.025565677343520388,0.025323824770775695,0.025014417531718607]	0.027087170042971113

only showing top 10 rows

feature test dataframe with vector range of 5 previous dates with $k = 50$

Define “range_5_previous_dates” as feature column for model parameter and “today_range” as a labels to predict column. The evaluate step is the same with training step but it use feature test data frame instead of using feature train data frame in training step, both of these data frames that was create after the lastest function in data preparation class in our source.

Finally, visualize the training MSE and test MSE on screen. The MSE can be change within adjust k previous days in the class data, default of k is 1 but user can adjust with how many days previous to predict the price of next day.



visualization of MSE for training and evaluating with $k = 50$

TASK 5 – Multi-class classification

5.1 Algorithm

Multi-layer Perceptron

Multilayer perceptron classifier (MLPC) is a classifier based on the feedforward artificial neural network. MLPC consists of multiple layers of nodes. Each layer is fully connected to the next layer in the network. Nodes in the input layer represent the input data. All other nodes map inputs to outputs by a linear combination of the inputs with the node's weights w and bias b and applying an activation function. This can be written in matrix form for MLPC with $K+1$ layers as follows:

$$y(\mathbf{x}) = f_K(\dots f_2(\mathbf{w}_2^T f_1(\mathbf{w}_1^T \mathbf{x} + b_1) + b_2) \dots + b_K)$$

Nodes in intermediate layers use sigmoid (logistic) function:

$$f(z_i) = \frac{1}{1 + e^{-z_i}}$$

Nodes in the output layer use softmax function:

$$f(z_i) = \frac{e^{z_i}}{\sum_{k=1}^N e^{z_k}}$$

The number of nodes N in the output layer corresponds to the number of classes.

MLPC employs backpropagation for learning the model. We use the logistic loss function for optimization and L-BFGS as an optimization routine.

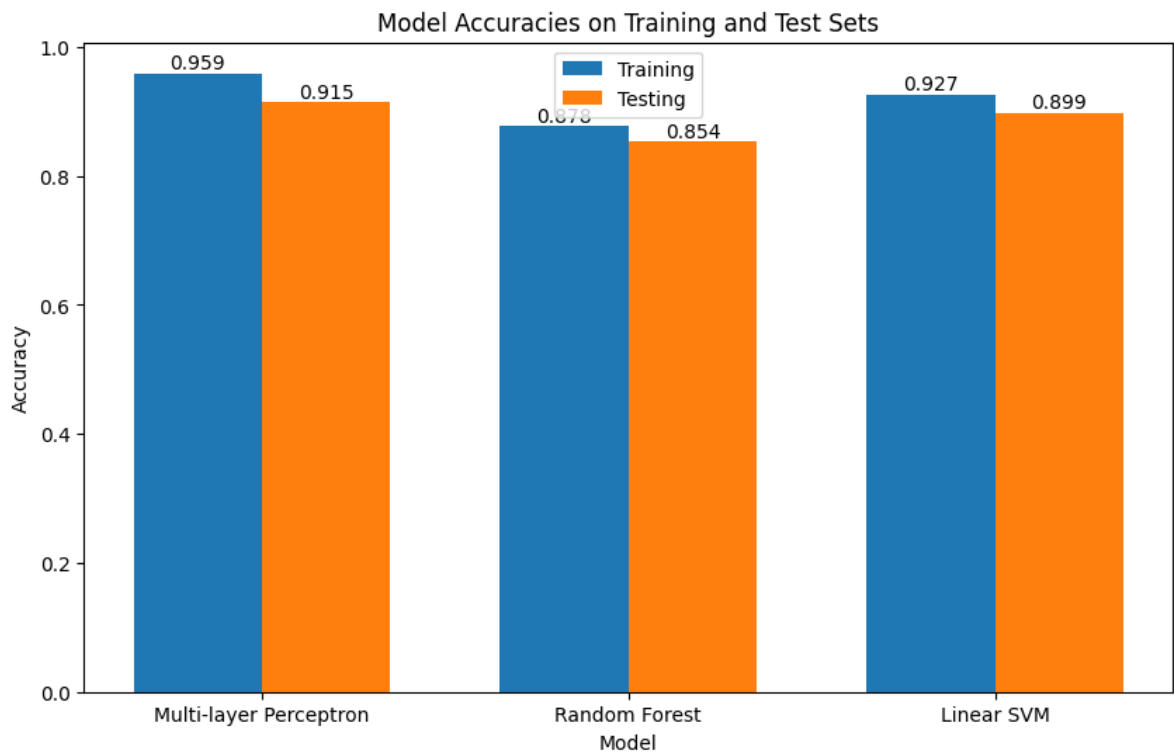
Random forest classifier

Random forests are a popular family of classification and regression methods. More information about the spark.ml implementation can be found further in the section on random forests.

Linear Support Vector Machine

A support vector machine constructs a hyperplane or set of hyperplanes in a high- or infinite-dimensional space, which can be used for classification, regression, or other tasks. Intuitively, a good separation is achieved by the hyperplane that has the largest distance to the nearest training-data points of any class (so-called functional margin), since in general the larger the margin the lower the generalization error of the classifier. LinearSVC in Spark ML supports binary classification with linear SVM. Internally, it optimizes the Hinge Loss using OWLQN optimizer.

5.2 Result



Model accuracies on training and test sets

To compare, the model with the highest accuracy rate on the MNIST-mini dataset is the multi-layer perceptron, with a "prediction" score of 0.959 for the training set and 0.915 for the test set. The model with the lowest accuracy rate is the Random Forest,

with only 0.878 for the training set and 0.854 for the test set. Finally, the Linear SVM model has a score of 0.927 for the training set and 0.899 for the test set.

REFERENCES

ENGLISH

- [1] KMeans," Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.clustering.KMeans.html> [Accessed: May. 15th, 2024].
- [2] “K-Means Clustering using PySpark Python”, GeekforGeeks.[Online]. Available: <https://www.geeksforgeeks.org/k-means-clustering-using-pyspark-python/> . [Accessed: May. 15th, 2024].
- [3] “SingularValueDecomposition”, Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.mllib.linalg.distributed.SingularValueDecomposition.html> . [Accessed: May. 16th, 2024].
- [4] “ALS" ,Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html> . [Accessed: Mar. 15th, 2024].
- [5] “LinearRegression" ,Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.regression.LinearRegression.html> . [Accessed: May. 15th, 2024].
- [6] “Multi-layer perceptron”, Apache Spark.[Online]. Available: <https://spark.apache.org/docs/latest/ml-classification-regression.html#multilayer-perceptron-classifier>. [Accessed: May. 20th, 2024].
- [7] “Random Forest”, Apache Spark.[Online]. Available: <https://spark.apache.org/docs/latest/ml-classification-regression.html#random-forest-classifier> . [Accessed: May. 20th, 2024].
- [8] “Linear Support Vector Machine:”, Apache Spark.[Online]. Available: <https://spark.apache.org/docs/latest/ml-classification-regression.html#linear-support-vector-machine> . [Accessed: May. 20th, 2024].

- [9] “What is good MSE value”, [Online]. Available: <https://stephenallwright.com/good-mse-value/> [Accessed: May. 20th, 2024].
- [10] “Multiple Linear Regression | A Quick Guide”. Scribbr [Online]. Available: <https://www.scribbr.com/statistics/multiple-linear-regression/> . [Accessed: May. 20th, 2024].
- [11] “The Complete Guide to Linear Regression Analysis”. Towardsdatascience (Medium). [Online] Available: <https://towardsdatascience.com/the-complete-guide-to-linear-regression-analysis-38a421a89dc2> [Accessed: May. 20th, 2024].